GITHUB SECURITY LAB · FINAL INTERVIEW

Prep Packet

Two talks · behavioral · research · questions

Rod Trent · print-and-go · generated October 07, 2026

1. **Speaker cheat sheet** — Part A — Presentations
2. **Dry-run & day-of checklist**
3. **Technical follow-up bank**
4. **Friday 11:00 — Kevin Backhouse** — Part B — Behavioral
5. **Behavioral drafts — Themes 1, 4, 8**
6. **Behavioral drafts — Themes 2, 9**
7. **Behavioral draft — Theme 6**
8. **Behavioral drafts — Themes 3, 5, 7, 10**
9. **Behavioral workbook (all themes reference)**
10. **GitHub Security Lab — research brief** — Part C — Research & questions
11. **Questions to ask them**

Talks: "CodeQL — Find the bug class, not just the bug" and "Breaking the chain — disrupting supply chain attacks on npm & GitHub Actions." Full decks (with speaker scripts in the notes), demos, and runbooks live in the repo: github.com/rod-trent/GHJ

PART A — PRESENTATIONS

Speaker cheat sheet

Two talks. Problem first, then tech, then demo, then takeaway. Each slide has full speaker notes in the .pptx — this is the one-page version for the day.

Talk 1 — CodeQL (12 slides, ~11 min scripted + interruptions)

Beat	Slides	~min
Title + problem: grep can't follow data flow; one bug class, many faces	1–2	1.5
Why it matters: one query protects everyone	3	0.5
Solution: code as data → QL query → results	4	0.75
Demo app "Octo Notes" + its deliberate blind spot	5	0.5
⏱ **Checkpoint: start slide 6 by ~4:00.** Past ~4:30 → you'll skip slide 10		
DEMO Act 1: default suite misses the SQLi	6	1.5
Models-as-data: one sink-model row	7	1.5
DEMO Acts 2–3: SQLi found + the QL query	8	1.5
Bridge: CodeQL also scans Actions (→ talk 2)	9	1.5
Optional: in the PR (skip if behind; don't sell Autofix)	10	0.75
Impact + takeaways	11–12	2

One-liner: *"Describe the bug once, catch it everywhere."*

Demo: captured run on the launchpad (Talk 1 → card 4 "Run output"), click ▶ per act. Say once that you captured it ahead of time.

Talk 2 — Supply chain (13 slides, ~13 min scripted + interruptions)

Beat	Slides	~min
Title + problem: self-replicating attacks — compromise → steal → propagate	1–2	1.25
Why it matters: you run far more code than you read	3	0.5
Strategy: break every link (the map)	4	0.75
⏱ **Checkpoint: slides 1–4 done by ~3:00**		
DEMO: install scripts — npm 11 runs it, npm 12 blocks it (protect this)	5–6	3
CI is the other door (script injection, pwn request) → CodeQL	7	1
Walk-through from the launchpad diff (Talk 2 → card 3); if asked to see it fire, show the captured Actions log (card 4)	8	1.5
Prevention: safe-by-default CI	9	1

Beat	Slides	~min
Prevention: kill the token (OIDC, staged publish, revoke)	10	1
Containment: Dependabot cooldown	11	0.5
Impact + takeaways	12–13	1.5

One-liner: *"Make the attack expensive at every step."*

One demo (slides 5–6): captured run on the launchpad (Talk 2 → card 4, **npm 11 vs 12**), click ▶ per act. Say once that you captured it ahead of time. Slide 8 is a **visual walk-through**, not performed. No live Actions injection in the interview; if asked, show the **Actions injection log** tab.

Key facts to have right (dates are 2026)

- **npm 12** (~July 2026): install scripts **off by default** (`allowScripts`); `--allow-git` , `--allow-remote` , etc. default to restricted. Needs **Node ≥ 24.15**.
- **actions/checkout** (enforced July 20, 2026): refuses untrusted fork checkout on `pull_request_target` / `workflow_run` unless you add `allow-unsafe-pr-checkout` .
- **Trusted publishing (OIDC)**: no long-lived `NPM_TOKEN` ; provenance auto-signed. CircleCI added April 2026.
- **Staged publishing** (`npm stage publish` , npm ≥ 11.15): human + 2FA approval before a package is installable; can be set "stage-only".
- **Dependabot cooldown** (July 2026): `cooldown:` in `dependabot.yml` , default delay for **version** updates; **security updates are never delayed**.
- **npm account protection** (June 2026): 72-hour read-only on email change / 2FA recovery.
- CodeQL workflow queries: `actions/code-injection` , `actions/untrusted-checkout` .

Likely Q&A

- *"Does modeling cause false positives?"* — Precision is tuned per sink kind; the demo shows the safe parameterized endpoint stays silent. You can also scope models and add `test` /sanitizer models.
- *"Does CodeQL need to build the code?"* — JS/TS and Actions use `build-mode: none` ; compiled languages need a build or autobuild.
- *"Is GitHub Advanced Security required?"* — Code scanning is free on public repos; GHAS for private. That's why the demo repo is public.
- *"Won't blocking install scripts break my build?"* — You approve the ones you need once (`npm install-scripts approve`), recorded in `package.json` . Most packages don't need them.
- *"Can't an attacker just name the flag / use a different sink?"* — The checkout flag is deliberately loud for review/CodeQL; new sinks are handled by adding a model (the talk's point).
- *"OIDC — what if I publish from somewhere other than GitHub/CircleCI?"* — Trusted publishing is per-provider; otherwise scope a short-lived token + staged publishing.

Dry-run & day-of checklist

Two talks; the demos are presented from captured runs on the launchpad. Work top to bottom. Boxes you can tick: `[ ]`.

T-minus 1 day — environment (do this on wifi, not at the venue)

- [ ] **Node version supports npm 12** — needs ≥ **24.15.0**: `bash node -v` If lower, upgrade (e.g. `nvm install 24` / install latest 24.x) and re-check. npm 12 still *blocks* on older Node but prints an engine warning on screen.
- [ ] **CodeQL CLI present** at `C:\Code\tools\codeql\codeql.exe`: `bash C:\Code\tools\codeql\codeql.exe version` If missing, download `codeql-bundle-win64.tar.*` from https://github.com/github/codeql-action/releases and extract to `C:\Code\tools\codeql`.
- [ ] **Deck build deps installed** (only if you plan to rebuild): `bash cd presentation/build && npm install --ignore-scripts`
- [ ] **gh authenticated** to the right account: `bash gh auth status`
- [ ] **Pre-cache npm 11 & 12** so Demo 1 runs offline and fast: `bash cd demos/supply-chain/npm-install-scripts && pwsh -File run-demo.ps1 -Prep`
- [ ] **Open the decks once** in PowerPoint; confirm Presenter View shows the speaker notes.
- [ ] **Full rehearsal** of both talks end to end, timed (target 12–13 min each + Q&A).

T-minus 1 hour — pre-flight at the venue

- [ ] Laptop charged + charger plugged in; screen-sleep / screensaver **off**.
- [ ] **Teams screen share, two monitors:**
- **Monitor A (shared):** the PowerPoint slideshow and the launchpad browser window, both maximized here.
- **Monitor B (yours):** Presenter View with the notes, plus Teams itself.
- PowerPoint → Slide Show → set **Monitor** to A and tick **Use Presenter View**.
- In Teams, Share → **Screen** → pick monitor A. Don't share the PowerPoint window (the audience won't see the browser when you switch) and don't use PowerPoint Live (no browser at all).
- Switching: **Alt+Tab** to the launchpad; to come back, **click the slideshow** on monitor A so the arrow keys drive the slides again. Your notes on monitor B stay visible the whole time.
- Launchpad in **audience view**, one tab per talk: `/?view=audience#codeql` and `/?view=audience#supply`.
- Browser zoom **~125%** so the terminal text survives Teams compression, then re-check that the replay lines aren't cut off on the right (see the replay check below).
- Teams/Windows notifications off (Do not disturb + Focus assist).
- Do a 2-minute test share with someone (or a second Teams client) and confirm they see both the slides and the browser.
- [ ] Browser logged in to GitHub; close noisy tabs, notifications, Slack/email.

- [ ] These tabs pre-opened:
- [ ] **Demo Launchpad**: https://rt-demo-launchpad.vercel.app (launch steps + the captured runs you present)
- [ ] If you'll screen-share it: open https://rt-demo-launchpad.vercel.app/?view=audience (or press **A**). The amber "Presenter view · A" badge in the header means coaching is showing; it disappears in audience view. Panic card (not linked anywhere): `/#panic`. Jump straight to a talk with `/#codeql` or `/#supply`.
- [ ] **Replay check, at the size you'll share:** step through every replay once in audience view, then hit **Reset** on each so they start at ▶ Build DB / ▶ Act 1 · npm 11 / ▶ Run log. Make sure long lines (CodeQL Act 2's line numbers, the npm warnings) aren't cut off on the right; if they are, widen the window.
- [ ] `github.com/rod-trent/octo-notes-demo` → **Security** → **Code scanning** (shows 7 open alerts: 5 code + 2 workflow; a 3rd was auto-closed by PR #2)
- [ ] The merged **PR #2** (the fix): `gh pr view 2 -R rod-trent/octo-notes-demo --web`
- [ ] The repo's **Actions** tab
- [ ] Decks open in **Presenter View**; one-pager PDF handy if anyone wants it.
- [ ] Verify network — or decide to run **fully offline** (both local demos work offline).

Talk 1 — CodeQL (demo pre-flight)

- [ ] **On stage you present the captured run** (launchpad, Talk 1 → card 4), not a live CLI run. Click ▶ through every act once so you know the rhythm. Fallback if the site won't load: `demos/backup/codeql-1..4-*.csv`.
- [ ] *Optional sanity check* (only if you want to confirm the real tool still matches the captured output, or expect a "can you run it?" follow-up after the talk): `bash cd demos/codeql && pwsh -File run-codeql-demo.ps1` You should see the same story as the replay: Act 1 misses the SQLi → Act 2 finds lines 11 and 20, line 29 stays silent → Act 3's query finds 11 and 20. Takes a few minutes. **Don't Ctrl+C a full run:** it wipes the database first. If that happens, run it again (or with `-SkipBuild`, which rebuilds it in ~1 min).
- [ ] Decide whether you'll also show the live alerts in the Security tab (needs network).

Run order on stage: slides 1–5 → **demo Act 1** (slide 6, captured run) → slide 7 → **demo Acts 2–3** (slide 8) → slides 9–12 → Q&A.

Talk 2 — Supply chain (demo pre-flight)

Demo 1 — npm install scripts (local, offline-safe) - [ ] `node -v` ≥ 24.15 (re-confirm on the venue machine). - [ ] Dry-run once: `bash cd demos/supply-chain/npm-install-scripts && pwsh -File run-demo.ps1` - [ ] Confirm: npm 11 runs the postinstall ("harvested … / would POST / would re-publish"); npm 12 **blocks** it. - [ ] **On stage you present the captured run** (launchpad, Talk 2 → card 4 → npm 11 vs 12). Click ▶ through Act 1, Act 2, Review once.

Slide 8 is a walk-through, not a live demo — you just talk over the vulnerable-vs-hardened slide. **Not firing the attack live in the interview**; if asked, show the captured Actions log (Talk 2 → card 4). The steps below are kept for reference only.

Live Actions injection (reference only, needs network) - [ ] **Enable the demo workflow** (it's disabled by default): `bash gh workflow enable "Issue triage (DEMO - vulnerable)" -R rod-trent/octo-notes-demo` - [ ] Have the payload issue title on the clipboard: `Login crash"; echo "PWNED as $(whoami)"; echo "Secret: $(echo $DEPLOY_TOKEN | base64)"; #` - [ ] Know where to click: Actions → **Issue triage (DEMO - vulnerable)** → latest run → *Greet the issue author* step → point at `PWNED as runner` + the base64 secret. - [ ] Fix comparison ready: `issue-triage-demo.yml` (vulnerable) vs hardened `issue-triage.yml` on `main`, plus the merged PR #2. - [ ] **Backup ready:** `demos/backup/actions-injection-run-log.txt` (a captured PWNED run).

Run order on stage: slides 1–4 → **demo** (slides 5–6, captured run) → slide 7 → **walk-through** (slide 8, not performed) → slides 9–13 → Q&A.

During each talk — reminders

- [ ] Problem first — don't jump to features.
- [ ] Narrate every demo step: what you're doing, why, what to watch for.
- [ ] Demos are **captured runs**: say once per talk that you ran them ahead of time. If the launchpad won't load, open `demos/backup/`, keep talking, move on. Don't debug on stage.
- [ ] **Driving the replays:**
- Click ▶ **once**, then wait until the output stops and the button shows the next step. A click while it's still printing does nothing.
- The ▶ button can **jump** (left to right) when its label gets longer. Look before each click.
- **Don't reload between CodeQL slide 6 and slide 8.** The replay is on the same card and picks up at ▶ Act 2; a reload starts it over at Build DB. (If it happens: Show all, then talk over Acts 2–3.)
- Keep the launchpad window **in front** while it plays. Browsers slow down timers in background windows, and the output crawls.
- [ ] Watch the clock in Presenter View. **Checkpoints:** CodeQL, start slide 6 by ~4:00 (past ~4:30 → skip slide 10). Supply chain, done with slides 1–4 by ~3:00.
- [ ] Supply chain slide 8: narrate from the **launchpad diff** (Talk 2 → card 3). If asked to see it fire, show the **Actions injection log** tab (Talk 2 → card 4); don't fire it live.
- [ ] Mention the cross-link once per talk (CodeQL ↔ supply chain = detection + prevention).

Teardown (right after Talk 2)

- [ ] **Disable the demo workflow** so it can't fire later: `bash gh workflow disable "Issue triage (DEMO - vulnerable)" -R rod-trent/octo-notes-demo`
- [ ] Close any demo issues you opened (closes every open issue whose title contains `PWNED`; PowerShell): `powershell gh issue list -R rod-trent/octo-notes-demo --state open --search "PWNED in:title" --json number --jq ".[].number" | ForEach-Object { gh issue close $_ -R rod-trent/octo-notes-demo -c "demo cleanup" }` Then confirm nothing is left: `gh issue list -R rod-trent/octo-notes-demo`
- [ ] Offer the one-pager (`presentation/One-Pager-Leave-Behind.pdf`) and repo link to the panel.

Panic card (if everything breaks)

- No network / launchpad won't load → walk the captured output in `demos/backup/` (`codeql-*.csv` , `actions-injection-run-log.txt`); skip the GitHub-UI beats.
- Laptop dies → the decks carry the full script in the notes; the one-pager PDF summarizes both talks.
- Repo: `github.com/rod-trent/GHJ` · Demo app: `github.com/rod-trent/octo-notes-demo`

Technical follow-up bank

Interviewers **will interrupt** and probe. The goal isn't to know everything — it's to reason well out loud and say "I don't know, here's how I'd find out" when you don't. Answers below are crisp talking points, not scripts.

Handling mid-demo interruptions (practice this)

- **Acknowledge, park, or answer briefly.** "Great question — short answer X; let me finish this step and I'll go deeper." Then actually circle back.
- **Never debug silently on stage.** If a command hangs >10–15s, narrate, switch to the backup file, keep the story moving. Your value is the reasoning, not the terminal.
- **If they challenge a claim**, welcome it: "Let's test that" beats defending. You can even run an ad-hoc variation if you know the repo cold.
- Keep a finger on the timer; a good interruption can eat 3 minutes.

Talk 1 — CodeQL

Q: What's actually in the CodeQL "database"? Is it just an AST? More than an AST. Extraction captures the AST plus a relational representation of the program: control-flow graph, the call graph, type information, and the scaffolding for data-flow/taint analysis. QL queries run against that relational model. That's why you can ask "does data flow from here to there," which pure syntax (AST/grep) can't answer.

Q: Difference between data flow and taint tracking? Data flow tracks values through assignments, calls, returns. **Taint tracking** is data flow plus "taint steps" — transformations that preserve attacker control even though the value changes (e.g., string concatenation, `JSON.parse`, templating). SQL injection needs taint tracking because the input gets reshaped on the way to the sink.

Q: Sources, sinks, sanitizers — and how do you cut false positives? Source = attacker-controlled input (`RemoteFlowSource`). Sink = dangerous operation (the SQL call). **Sanitizer/barrier** = a node that makes data safe (e.g., parameterization, an escaping function); you add barriers to the config so paths through them aren't reported. That, plus precise sink modeling, is how you tune precision. In the demo, the parameterized endpoint is clean precisely because a prepared statement isn't a string-concatenation sink.

Q: How is "models as data" different from writing a query? Why is it safer? A query is logic; a model is *data* describing a library's sources/sinks/summaries in YAML. Adding a model teaches **every existing query** about a new library without touching query logic — lower blast radius, easy to review in a PR, versioned, and shippable via `.github/codeql/extensions` where code scanning auto-loads it. Writing a new query is for a new *kind* of bug; modeling is for new *libraries/APIs* exhibiting a known kind.

Q: Could an attacker evade your model by using a different method/alias? Yes — models are as complete as you make them. That's the honest limitation and the reason the bounty/community model exists: you broaden coverage by modeling more entry points, and MRVA helps you discover where real code uses them. It's an arms race you win with breadth + community.

Q: Does CodeQL need to build the code? JS/TS, Python, Ruby, and Actions use `build-mode: none` (no build). Compiled languages (Java/Kotlin, C/C++, C#, Go, Swift) need the build observed — autobuild or a manual build — because extraction hooks the compiler.

Q: False negatives — what does CodeQL miss? Unmodeled libraries/sinks (the whole point of the demo), dynamic reflection/ `eval`, flows through serialization it can't follow, and anything outside the analyzed languages. It's a strong net, not a proof of absence.

Q: Precision vs. recall — how do suites decide what's on by default? `default` < `security-extended` < `security-and-quality`. Higher coverage trades in more noise. Queries carry `@precision` and `@problem.severity`; the default suite favors high-precision, high-severity so developers trust the signal. You enable broader suites where a team can absorb the noise.

Q: How does Copilot Autofix relate to CodeQL? CodeQL localizes the vuln and provides the data-flow context; Autofix uses that context to propose a concrete code change with an explanation. CodeQL is the ground truth; the LLM writes the patch suggestion. Human still reviews/merges.

Q: How does CodeQL analyze Actions YAML? There's an `actions` extractor and dedicated query packs (`actions/code-injection`, `actions/untrusted-checkout`, etc.). Same engine, workflows as the "program," expression contexts like `${{ github.event.* }}` as sources and `run:` as the injection sink.

Q: MRVA? Multi-Repository Variant Analysis — run one query across thousands of repos from VS Code to find every instance of a pattern ecosystem-wide. It's "bug class, not bug" at ecosystem scale, and a core Security-Lab workflow.

Talk 2 — Supply chain

Q: How exactly does OIDC trusted publishing remove the token? Instead of a long-lived `NPM_TOKEN` stored in CI, the CI job requests a short-lived **OIDC ID token** from GitHub (`id-token: write`). npm verifies that token against a **trusted publisher** config you set on the package (this repo, this workflow). No secret at rest = nothing to exfiltrate, and the token is minted per-run and expires. Bonus: it enables **signed provenance** automatically.

Q: What's provenance / how does it relate to Sigstore and SLSA? Provenance is a signed attestation of *where and how* a package was built (repo, commit, workflow). npm's provenance uses **Sigstore** for signing/transparency. It moves you up the **SLSA** build-integrity levels. It doesn't stop a malicious maintainer, but it makes tampering and "typosquat built from nowhere" detectable.

Q: If install scripts are off by default, won't builds break? Some packages legitimately need them (native modules via `node-gyp`). You review the blocked list once (`npm install-scripts ls`), approve the few you trust (`approve`), and that decision is recorded in `package.json`. Most packages don't need scripts, so the default is safe and the exceptions are explicit and auditable.

Q: Cooldown vs. pinning vs. lockfiles — why cooldown? They're layers. **Lockfiles** pin exact versions you've already chosen. **Pinning actions to a SHA** stops a mutable tag being swapped. **Cooldown** addresses a different gap: the *moment you adopt a brand-new release*. Most malicious versions are caught within hours–days, so waiting lets ecosystem detection win the race — and **security updates are exempt**, so you don't delay real fixes.

Q: Why is `pull_request_target` dangerous when `pull_request` isn't? `pull_request` runs with a **read-only** token and no secrets for fork PRs. `pull_request_target` runs in the **base** repo's context — secrets + write token — but at the fork's behest. If you then check out and execute fork code, untrusted code runs privileged. The 2026 checkout change refuses that by default unless you add the deliberately-loud `allow-unsafe-pr-checkout`.

Q: What was Shai-Hulud / tj-actions — be accurate. Reference the *pattern*, not invented numbers. **tj-actions** (2025): a popular Action was compromised and modified to dump CI secrets into logs across many repos. **Shai-Hulud**: a self-replicating **npm worm** that used stolen publish tokens to inject itself into more packages — the canonical "compromise → steal → propagate" loop. If you're unsure of a figure, say "on the order of" or defer.

Q: Does OIDC work outside GitHub/CircleCI? Trusted publishing is per-provider (GitHub Actions, CircleCI, …). Elsewhere you fall back to a short-lived, tightly-scoped token plus **staged publishing** (human + 2FA gate) so compromised CI still can't release unilaterally.

Q: Account-protection detail? High-impact npm accounts enter a **72-hour read-only mode** after an email change or 2FA recovery — blunting phishing-based takeover before the attacker can publish.

Q: How does this connect to CodeQL (consistency)? Prevention (these defaults) + detection (CodeQL's `code-injection` / `untrusted-checkout`). Same philosophy as Talk 1: encode the defense once so everyone benefits.

If you get a question you can't answer

Say so, then show reasoning: *"I don't know off-hand. My mental model says X, so I'd expect Y — I'd verify by [reading the extractor docs / testing it in the demo repo]."* That's a **better** signal than bluffing, especially for a research team.

PART B — BEHAVIORAL

Friday 11:00 — Culture & Behavioral with Kevin Backhouse

Read this the night before and again at 10:30. It **adapts the existing behavioral drafts** (05–08) to the person across the table. Stories don't change; the *framing and the hooks* do.

> **Who he is, in one breath:** a career vulnerability researcher, ex-Semmle (he wrote CodeQL's C/C++ range-analysis and value-numbering libraries), at GitHub Security Lab since it launched in 2019. Based in the UK, Oxford-educated. He is likely the **manager of the Lab's security research sub-team** (TheOrg lists him that way, and in mid-2026 he posted "I'm hiring two Staff Security Researchers for *my team*"). He's also the **security contact for Exiv2**, so he sees disclosure from the *maintainer's* side too.
>
> **What that means for you:** he's probably not assessing whether you can out-research him. He's assessing **whether you'd be a great partner to his researchers**: someone who can turn their findings into community impact, who respects the craft, and whom he'd enjoy working with asynchronously across an ocean. *Assumption:* the guide says this slot is Culture & Behavioral. Expect him to probe technical claims anyway, because he's a researcher.

1. His work: know these cold (one line each)

When	What	Why it's memorable
2018 (Semmle)	**CVE-2018-4407**, XNU ICMP heap overflow	One packet on the LAN could crash any Mac/iPhone. Found with QL.
2018	Apple **packet-mangler RCE**, **NFS diskless-boot** overflows, **librelp** (rsyslog) stack overflow	His LinkedIn articles. librelp was a *collaboration* write-up with the maintainer.
2018	"Using QL to find kernel stack buffer overflows in **Qualcomm MSM**"	Variant analysis before MRVA existed.
Nov 2020	**"How to get root on Ubuntu 20.04 by pretending nobody's /home"**	accountsservice DoS → gdm3 runs initial setup → new admin. **No memory corruption.** Famous.
Jun 2021	**polkit CVE-2021-3560**	A 7-year-old auth bypass: kill `dbus-send` at the right moment → root on most distros.
Dec 2021	"Getting root on Ubuntu through wishful thinking"	Another accountsservice LPE.
Nov 2021	**Exiv2 "blue-teaming" series**	Advisory process, **three rules of bug fixing**, custom CodeQL in code scanning, OSS-Fuzz.
Oct 2023	**CVE-2023-43641**, 1-click RCE on GNOME (libcue)	Just *downloading* a `.cue` file was enough, because tracker-miners auto-indexes ~/Downloads.
Dec 2023	"Cueing up a calculator"	A *teaching* post on Linux exploit development using that bug.

When	What	Why it's memorable
Feb 2024	Repo-jacking explained	Supply-chain risk from renamed/deleted accounts. Ties to **your Talk 2**.
Jan 2026	**"Community-powered security with AI"**: launch of the **Taskflow Agent**	Open-source YAML "taskflows" + MCP toolboxes (incl. CodeQL). He called it "a tool we want to use ourselves."
Apr 2026	**"Finding a 16-year-old bug isn't as surprising as you think it is"** (LinkedIn)	See §2. **Read it before Friday.**

His **three rules of bug fixing** (Exiv2 series): add a regression test, fix the bug, and **find and fix the variants**. "If you find one bug, there are almost always others nearby." This is **literally your Talk 1** (the second SQLi on `/by-owner` is the variant). Use that line.

Don't confuse: the 2023 Exim CVEs were ZDI's, not his. Don't credit him with them.

---

2. His April 2026 article: have a real opinion

His argument, in brief: People are fixating on the *age* of bugs AI finds (e.g. the 16-year-old FFmpeg H.264 bug from Anthropic's Project Glasswing). That's expected: - Security research has always meant **looking at code nobody else looked at**: error paths and corner cases, off the "happy path." - FFmpeg is about 2M lines and already heavily fuzzed. Fuzzing misses bugs when hitting the path by chance is improbable (a frame with *exactly* 65,536 slices). **AI automates careful reading** and finds what slipped through. - The DEF CON pattern: the hard part of most talks is *reverse engineering to reach the code*; the bugs themselves are often trivial. - Maintainers are getting **a flood of AI-generated reports**: "beneficial in the longer term, but it's absolutely not fun in the short term." So the Lab **open-sources its AI tooling** so maintainers can find problems themselves, on their own schedule. - He ends by praising **FFmpeg's maintainers**. His respect for maintainers runs through all his work.

Your take (say it in your words):

> "What struck me was the maintainer point at the end. The tooling is getting good fast, but the bottleneck has moved from *finding* bugs to *people absorbing them*. That's an education and advocacy problem as much as a research one: teaching maintainers to run these tools *before* someone else does, and teaching AI-assisted reporters what a report worth a maintainer's time looks like. Verified, reproducible, ideally with a patch. That's work I'd love to own."

That connects his concern directly to the DevRel role. **Context to know:** GitHub restructured its own bug bounty in July 2026 (an invite-only VIP tier, and new researchers capped at four submissions) partly because of AI-report volume.

---

3. What he's been amplifying (his LinkedIn, last ~6 months)

- **AI-powered fuzzing with the Taskflow Agent** (Antonio Morales, `seclab-taskflows-fuzzing`). The agent picks targets, writes harnesses, runs AFL with doubling time budgets, and triages crashes.
- **"How to scan for vulnerabilities with GHSL's open-source AI framework"** (Man Yue Mo & Peter Stöckli, Mar 2026). It runs threat modeling, then issue suggestion, then a fresh-context audit with

strict evidence rules. Across 40+ repos: **1,003 suggested → 139 flagged → 19 reportable after manual review.** That funnel is a great talking point on *human verification*.

- **GitHub Secure Open Source Fund**: Session 5 is open ($10k + 3-week training + Lab office hours). Session 4 had 50 projects and 71 maintainers, and 92% finished with secret scanning, code scanning, private vulnerability reporting (PVR) and Dependabot on.
- **"Repository Under Attack" red-team workshop** for Fund maintainers: a simulated supply-chain incident (malicious package, exposed Actions secrets). **That's your Talk 2, as a workshop.**
- **actions/checkout v7** refusing pwn-request checkouts (`allow-unsafe-pr-checkout` escape hatch). Already in your Talk 2. "Secure by default doesn't always mean removing a behavior."
- **7-Zip GHSL-2026-140 / CVE-2026-48095** PoC, and Jaroslav Lobačevski's talk "LLM-assisted vulnerability hunting: hype vs. reality."
- **His hiring posts**: two Staff Security Researchers, remote (USA + Germany). His team is **fully remote and multi-timezone**, so Theme 7 (async) matters to him personally.

The pattern: AI-assisted research tooling, open-sourced for maintainers, plus supply-chain hardening and maintainer education. Your two talks sit squarely in this.

---

4. Re-aim your stories for Kevin

Theme	Draft	Re-aim it like this
1 · Why the Lab	05	Add: *"Your team's research is the raw material. My job would be to make sure it reaches the 500:1 developers who'll never read an advisory."* Name **one** specific Lab artifact you'd advocate (Taskflow Agent or the Fund).
2 · Curiosity / OSS	06	Lead with **what you built for this loop**: a vulnerable app, a model pack, a custom taint query, a live Actions injection. Mention you **read the Taskflow audit post** and what struck you (the 1,003 → 19 funnel). Be honest that you're a *translator*, not an exploit developer.
4 · Technical decision	05	Fine as-is. If he probes, have one technical detail from building the demos ready (§5).
5 · Helping others	08	Researchers often hate writing for broad audiences. Frame it as *"helping an expert get their work understood,"* which is exactly what his team needs from DevRel.
6 · Learning	07	**Strongest fit.** Learn it → build it → teach it is Kevin's own pattern ("Cueing up a calculator" teaches exploit dev using his own bug).
7 · Remote/async	08	He manages a UK/US/Germany team. Stress written artifacts, issue/PR culture, and **knowing when sync is worth it**.
8 · Ownership	05	Fine.
9 · Excellent work	06	Tie to his **three rules**: excellent = fix the class, not the instance, plus a regression test so it stays fixed. Your Talk 1 thesis.
10 · Failure	08	Needs a real one. He'll value an honest **technical** miss over a polished story.

---

5. Technical probes he might throw (have a crisp answer)

- *"How did you get CodeQL to find the node:sqlite injection?"* → It wasn't modeled. I added a `sinkModel` extension: `node:sqlite` → `DatabaseSync` instance → `prepare/exec` arg 0 as `sql-injection`. Then the stock `js/sql-injection` query lit up both variants and stayed silent on the parameterized one.
- *"Why is the Actions injection exploitable?"* → `${{ }}` is template-expanded **before** the shell runs, so the title becomes script text. The fix is an env var (`$TITLE` is data), no secrets in that job, and a minimal token.
- *"Where does CodeQL fall short?"* → Unmodeled libraries (your own demo shows it), dynamic dispatch, and logic bugs with no clear source-to-sink shape. **His polkit and gdm3 bugs are exactly that kind.** Saying so shows you know where the craft lives.
- *"Fuzzing vs. static analysis vs. AI?"* → They're complementary. Fuzzing needs a probable path, static analysis needs a model of the bug class, and AI reads code like a patient human but needs **verification** (the 1,003 → 19 funnel).
- **If you don't know: say so, then reason out loud.** A researcher respects "I haven't done exploit dev at that level; here's how I'd approach finding out" far more than a bluff.

---

6. Questions for Kevin (pick 3; they're built to start a conversation)

1. "In your Glasswing piece you said the AI report flood is *'absolutely not fun in the short term'* for maintainers. From the Exiv2 side, **what does a report worth your time look like**, and what would you want every AI-assisted reporter to learn first?"
2. "The audit-taskflow post went 1,003 suggestions → 19 reportable. **Where does a human researcher add the most value in that funnel today**, and do you expect that to move?"
3. "You've written some of the Lab's best *teaching* posts, like 'Cueing up a calculator.' **How do you decide when a finding deserves a tutorial and not just an advisory?** And how could a Developer Advocate make that cheaper for your researchers?"
4. "Your three rules of bug fixing end with 'find the variants.' **How often do maintainers actually do that third step**, and what gets in the way?"
5. "Your team is spread across the UK, US, and Germany. **What does a great working relationship between your researchers and a Developer Advocate look like day to day?**"
6. "What's a piece of the Lab's research you think **deserved far more attention than it got**?"

Rapport openers if there's a natural moment (don't force it): his team-page bio ("catching up on all the hacking I should have done in the 1990s"), or the kids' spelling-practice Python project on his GitHub. Both are about *teaching people to code*, which is your whole career.

---

7. Avoid

- Overclaiming research depth. He'll know. Your edge is **translation at scale**, so own it.
- Treating AI as magic **or** as hype. His position is nuanced: a real revolution that needs verification and maintainer empathy.
- "GitHub is a great company." Name the Lab's **mechanism**: encode expertise (queries, taskflows), open-source it, and mobilize the community.
- Attributing the Exim CVEs or anything you can't source to him.

Sources: Kevin's LinkedIn profile & activity (viewed Oct 6, 2026), his LinkedIn article (Apr 10, 2026), github.blog/author/kevinbackhouse, securitylab.github.com/team, the Taskflow Agent and audit-framework posts on github.blog. The "manager of the research sub-team" title comes from TheOrg plus his hiring post. It's an inference, so don't state it as fact to him.

Behavioral drafts — Themes 1, 4, 8 (from public background)

How to use this: these are first-draft scripts built from your *public* record (Microsoft Cloud Security Advocate; Must Learn KQL & Must Learn AI Security; myITforum; Windows IT Pro; SANS MS; 30+ yrs speaking/writing). Everything tagged `[VERIFY]` is my inference — correct or cut it. Each runs ~2 minutes. Say "I," land the outcome, end on the Reflection. Swap in real numbers wherever you see `[VERIFY: metric?]` — a concrete figure makes these much stronger.

Sources for the public facts: your Microsoft advocate page, rodtrent.substack.com, the Must Learn KQL GitHub course, and the Definitive Guide to KQL.

Theme 1 — Why security, why GitHub, why the Security Lab

> Structure: a career *thread*, not a résumé. The thread is: **I've spent my career making hard security topics approachable and teaching them at scale.** That IS developer advocacy.

Draft (spoken):

"My whole career has been one thread, even when the technology changed under it: taking something complex and making it approachable for a huge audience. I started by building and running IT communities — myITforum — and editorial teams `[VERIFY: editor-in-chief, Windows IT Pro]`, where the job was translating hard systems problems for practitioners. Then I went deep on security at Microsoft — Sentinel and KQL — and I noticed people were drowning in the *data* side of defense. So I wrote **Must Learn KQL**, an open, free, serialized course on GitHub, and it became one of the default on-ramps for learning KQL `[VERIFY: scale — downloads/translations/book sales?]`. When generative AI arrived, I did it again with **Must Learn AI Security** — 400-plus pages on securing AI `[VERIFY: exact length/outcome]`.

So why security? Because defenders are permanently outnumbered, and the only thing that scales is *teaching* — turning one expert's knowledge into something thousands of people can use. Why open source specifically? Everything I'm proudest of, I gave away on GitHub, because reach and community contribution beat a paywall every time.

And that's exactly why the **Security Lab**, not just GitHub. The Lab's entire premise is that you can't out-hire this problem — you scale expertise through the community: CodeQL queries anyone can write, the query bug-bounty that turns one researcher's insight into protection for everyone, MRVA, the Advisory Database. That's the thing I've been doing for 30 years, pointed at open-source security. A **Developer Advocate** role there isn't a pivot for me — it's the same job I've always loved, with a bigger lever."

Reflection line: "The lesson I keep relearning: the highest-impact security work isn't the bug I personally find — it's the explanation that lets a thousand other people find it."

Theme 4 — A hard technical decision

Problem → Options → Trade-offs → Decision → Outcome → Learning

> Grounded in the **Must Learn KQL** build. If you have a meatier *security-architecture* decision from a large Sentinel engagement, swap it in — but this one is clean, documented, and shows product/technical judgment, which fits a DevRel role well.

Draft (spoken):

"**Problem:** KQL was becoming essential for defenders on Sentinel, but the learning curve was a wall. Microsoft had reference docs, but no approachable on-ramp, and people were bouncing off it. I decided to fix that — and the real decision was *how* to build and deliver it so it would actually scale and last.

Options I weighed: one, write a traditional book or a paid course — polished, but slow, closed, and stale the day it ships. Two, a stream of blog posts — fast and free, but unstructured and un-versioned. Three `[VERIFY: framing]`, treat the curriculum like an open-source *project*: host it on GitHub, serialize it in small, teachable parts, version it, and let the community file issues and PRs — even translations `[VERIFY: were there community translations?]`.

Trade-offs: option three meant no revenue and an ongoing maintenance burden I'd carry personally, and publishing in the open where mistakes are visible. But it maximized reach, let the material stay current, and — the part I cared about — let the community *improve* it.

Decision: I went with the open, versioned, serialized approach, and later added an **assessment** `[VERIFY: Must Learn KQL Assessment, ~2022]` so people could prove they'd learned it.

Outcome: it became one of the go-to ways people learn KQL `[VERIFY: metric — stars, downloads, book, adoption?]`, and later a published book `[VERIFY]`.

Learning: treating *education* like open-source software — versioned, in the open, community-editable — compounds in a way a static artifact never does. I reused that exact playbook for Must Learn AI Security."

Reflection line: "I now default to building teaching artifacts *as* open-source projects, not products — the community turns them into something better than I could alone."

Theme 8 — Ownership in ambiguity

> Their desired arc: **recognized the gap → created clarity → took ownership → brought others along → improved it → applied the learning.** Built from **Must Learn AI Security** (emerging, genuinely ambiguous space). Alternative: **myITforum** is an even purer "I owned a whole community for ~20 years" story — use that if they want longer-horizon ownership.

Draft (spoken):

"**The gap:** when generative AI took off, security teams were being asked to secure something nobody had a playbook for. The space was pure ambiguity — prompt injection, model risks, data leakage — and there was no clear owner of 'how does a normal defender actually get started here.' `[VERIFY: your framing of the moment, ~2023]`

Creating clarity: rather than wait for someone to define it, I started writing **Must Learn AI Security** in the open — breaking a chaotic topic into small, concrete, teachable pieces, one at a time, published where anyone could read and respond. `[VERIFY: cadence]`

Taking ownership / bringing others along: I committed to it publicly and kept shipping, which pulled in feedback, corrections, and real-world questions from practitioners `[VERIFY: community response`

`specifics]`. The series grew past **400 pages** and became a formal, organized resource `[VERIFY: book/compendium]` — and it fed directly into my work explaining tools like Microsoft Security Copilot `[VERIFY]`.

Improvement + what I carried forward: I turned an ambiguous, intimidating space into an on-ramp a lot of defenders actually used `[VERIFY: metric?]`. What I carried forward: in a space this ambiguous, *publishing early and iterating in the open* creates clarity faster than trying to design the perfect taxonomy first. Shipping the v1 is what recruits the collaborators."

Reflection line: "Ambiguity doesn't get resolved by waiting for ownership to be assigned — you create clarity by putting a rough, honest v1 in the open and letting people build on it."

Notes on using these

- **Fill the `[VERIFY]` and `[VERIFY: metric?]` tags** — a real number (downloads, pages, adoption, engagement) turns a good answer into a memorable one.
- **Don't memorize word-for-word.** Learn the beats; speak it fresh so it sounds like you.
- **Tie back to the Lab** in Theme 1 and (lightly) Theme 4 — the "scale expertise through community / open source" thread is your single strongest fit, and it mirrors both your talks.
- Want me to also draft **Theme 2 (OSS/curiosity)** and **Theme 9 (definition of excellent work)**? Those are easy wins from the same material.

Behavioral drafts — Themes 2 & 9 (from public background)

Same rules as the T1/T4/T8 file: built from your public record, `[VERIFY]` = my inference to confirm or cut, `[VERIFY: metric?]` = drop in a real number. ~2 minutes each. Say "I," end on the Reflection. Keep the stories *distinct* from Themes 1/4/8 (don't retell Must Learn KQL as a decision here — use the angles below).

Theme 2 — Security, open source & technical curiosity

> This one isn't a single STAR story — they want to see *genuine* engagement: security work, OSS involvement, how you stay current, and **what you're exploring now + what you're doing with it.** Lead with curiosity as a habit, then give concrete proof.

Draft (spoken):

"Curiosity is kind of my operating system — I learn something by teaching it in public, and I do it out loud on GitHub and my blog so other people can build on it.

On the **security** side, most of my hands-on work has been in detection and threat hunting — KQL, Sentinel, helping teams actually interrogate their security data `[VERIFY: largest Sentinel implementations]`. On the **open-source** side, the things I'm proudest of are public repos, not internal docs — Must Learn KQL, Must Learn AI Security — because an open repo invites issues, PRs, and translations `[VERIFY]`, and it keeps improving after I stop touching it.

How I **stay engaged** with the community: I publish constantly `[VERIFY: Substack cadence]`, I speak at conferences `[VERIFY: 30+ years / which ones]`, and I show up on podcasts `[VERIFY: Practical 365, Microsoft Threat Intelligence]`. I treat that two ways — as teaching, and as a listening post for what practitioners are actually stuck on.

What I'm **exploring right now** is the collision of **AI and security** — both securing AI systems (prompt injection, model risk — the Must Learn AI Security work) and using AI to make defenders faster, like Security Copilot `[VERIFY]`. And more recently I've gone deep on **CodeQL and static analysis** — 'code as data,' models-as-data, how you describe a whole bug class once and catch every instance `[VERIFY: your framing]`. What I'm *doing* with it is exactly what the Security Lab does: turning what I learn into something other people can pick up and run with — which is why the Lab's AI-plus-static-analysis direction, like the Taskflow Agent, genuinely excites me."

Reflection line: "I've learned I don't really understand something until I've taught it in the open and had the community poke holes in it — so 'learning' and 'advocacy' are the same activity for me."

Theme 9 — My definition of excellent work

> Have a *philosophy* **and** a real example where you hit the bar and knew it. Pick a different lens than Theme 4: there, Must Learn KQL was a *decision*; here, make it about a *quality bar* and how you *measured* it.

Draft (spoken):

"**My definition:** good work is correct; excellent work is correct *and* understood and reusable by the next person — it solves the real underlying problem, it's maintainable, and you can actually *measure* that it landed. In my world, the last part matters most: it's easy to produce content that looks great and teaches nothing. Excellent means the audience can go *do* the thing afterward.

An example: with Must Learn KQL I could see people reading it, but 'read' isn't 'learned.' So I held myself to a higher bar — I built an actual **assessment** `[VERIFY: Must Learn KQL Assessment, ~2022]` so learners could *prove* they could write KQL, and so I'd get a real signal about where the material was failing. That did two things: it turned a pile of articles into something with a measurable outcome, and the results told me exactly which sections to rewrite `[VERIFY: what the data showed / pass rates?]`. I iterated on the weak spots instead of guessing.

Why that's my bar: understanding the real problem (people needed to *apply* KQL, not admire it), maintainability (open and versioned so it survives me), communication (small, teachable pieces), and a measurable outcome (the assessment). When all four line up, I know it's excellent — not because it's polished, but because it *changed what someone could do* `[VERIFY: adoption/impact metric?]`."

Reflection line: "I stopped trusting 'this looks great' as a quality signal years ago — now I ask 'how will I know it actually worked?' before I ship, and build that measure in from the start."

Notes

- **Theme 2** lets you name **CodeQL / Taskflow Agent** as live curiosity — a strong, authentic "why the Lab" reinforcement. Keep it honest: frame it as recent, genuine exploration.
- **Theme 9's** measurable-outcome angle (the assessment) is distinct from Theme 4's decision angle, so you won't sound repetitive if both come up.
- Fill the `[VERIFY: metric?]` tags — "excellent work" answers are far stronger with a number.
- Remaining themes with no draft yet: **3** (collaboration/different perspectives), **5** (helping others — two-way), **6** (learning & growth), **7** (remote/async), **10** (a failure). Those need a real anecdote from you; the workbook has prompts. Want me to draft any once you give notes?

Behavioral draft — Theme 6: Learning & growth (from public background)

Same rules: `[VERIFY]` = my inference to confirm/cut, `[VERIFY: metric?]` = drop in a real number. ~2 minutes. They stress: **what are you *actively* learning, and what are you *doing* with it?** Concrete beats general. Keep this distinct from Theme 2 — there you talked about curiosity as a habit and community engagement; **here, focus on your learning *method* and *trajectory*, and the learning→output loop.**

Draft (spoken):

"The way I grow is deliberate: I pick a hard area, commit *publicly* to teaching it, and let that deadline force me to actually master it. Publishing is my study method — if I can't explain it simply in the open, I don't really know it yet.

You can see that as a trajectory. I went deep on **systems management**, then committed to **security data and threat hunting** — KQL and Sentinel — and taught my way to fluency with Must Learn KQL. I even went back for formal depth: a **master's in Information Security Engineering from SANS** `[VERIFY: MS, SANS Technology Institute]`, because I didn't want to be only self-taught in security. Then generative AI arrived and I retooled again into **AI security** — prompt injection, model risk, securing AI systems — and turned that into Must Learn AI Security `[VERIFY: 400+ pages]`.

What I'm actively learning right now is two things. One, **AI *for* security** — how tools like Security Copilot change a defender's workflow `[VERIFY]`. Two — and this is recent and directly relevant — **CodeQL and static analysis**: 'code as data,' writing queries, models-as-data, the idea that you describe a bug class once and find every instance `[VERIFY: your framing]`.

And here's what I'm *doing* with it — which is the part I care about most. I don't learn CodeQL to have learned it; I built a small vulnerable app, wrote the queries and a model, and turned the whole thing into a teachable demo and write-up `[VERIFY: your interview project — keep or soften as appropriate]`. That's my loop every time: learn it, build something real with it, then package it so someone else gets there faster. It's exactly the Security Lab's model — expertise only matters once you've made it reusable."

Reflection line: "The biggest growth lesson of my career: the fastest way to *learn* something is to commit to *teaching* it before you're ready — the obligation pulls the mastery forward."

Notes

- The CodeQL line is genuinely true for you right now (you built exactly that for this loop) — it's a strong, authentic "I practice what I'm interviewing for" signal. Soften the project reference if you'd rather not foreground that it was interview prep; "I built a small vulnerable app and modeled it in CodeQL" stands on its own.
- Distinct from Theme 2: Theme 2 = curiosity *habit* + community; Theme 6 = learning *method* + *trajectory* + the learn→build→teach loop. If both come up, lead each with its own angle.
- Fill the `[VERIFY]` tags — especially the SANS degree and the AI Security page count.

- Still undrafted (need your anecdote): **3** collaboration/disagreement, **5** mentoring (two-way), **7** remote/async, **10** a failure. Paste a few bullets for any and I'll match the format.

Behavioral drafts — Themes 3, 5, 7, 10 (scaffolds from public background)

> **Read this first.** These four are more inferred than the earlier drafts. Themes 3, 5, and 10 depend on *specific interpersonal moments* I can't know, so treat them as **scaffolds** — keep the shape, swap in a real memory. Theme 7 (remote/async) is on firmer ground because it's genuinely your operating model. Everything `[VERIFY]` is my inference; `[REPLACE]` means "use your own anecdote here." ~2 min each, STAR + Reflection, say "I."

Theme 3 — Collaboration & different perspectives

> Best public anchor: **co-authoring *The Definitive Guide to KQL*** with other Microsoft security experts `[VERIFY: co-authors, e.g. Mark Morowczynski / Matt Zorich]`. Multiple strong authors with different teaching instincts is a clean "worked with people who approached it differently" story. If a sharper disagreement comes to mind, `[REPLACE]`.

Draft (spoken):

"When I co-authored a book on KQL with several other security experts `[VERIFY]`, we hit exactly this. I come at teaching from a *make-it-radically-approachable, smallest-possible-steps* angle — that's the Must Learn style. One of my co-authors `[VERIFY]` wanted far more depth and rigor up front, worried that simplifying would leave readers under-prepared for real threat hunting.

My first instinct was to defend my approach. Instead I made myself actually argue *their* side — I went and looked at where beginner-friendly material leaves people stranded later, and they had a real point. So rather than pick a winner, we restructured `[VERIFY]`: approachable on-ramps first, then deliberate 'now go deeper' sections, so the same book served the newcomer and the practitioner. I owned the on-ramp chapters; they owned the depth.

What changed because of the collaboration: the book was stronger and reached a wider audience than either of our instincts alone would have `[VERIFY: outcome/reception]`. And I changed — I stopped treating 'simple' and 'rigorous' as a trade-off and started designing for both."

Reflection: "I learned to steel-man the other person's approach *before* defending mine — most of the time the best answer is a structure that holds both, not a winner."

Theme 5 — Helping & developing others (two-way)

> Anchor: mentoring a newer community contributor / writer — natural given your editorial and community-leadership past `[VERIFY: myITforum / Windows IT Pro / MVP mentoring]`. The team wants it to be **two-way** — you learned something too. `[REPLACE]` with a specific person if you can.

Draft (spoken):

"Through the communities and editorial teams I ran `[VERIFY]`, a lot of what I did was help practitioners become *writers and speakers* — people who were brilliant hands-on but convinced they had nothing to teach.

I worked with one contributor like that `[REPLACE: real person//situation]` — strong technically, terrified to publish.

What I did: I didn't edit their work *for* them, I co-published the first one *with* them and showed my own messy drafts so they'd see that 'polished' is a process, not a talent. Then I pushed them to publish solo and just amplified it.

The result: they kept going `[VERIFY: became a regular contributor/speaker?]`. But the two-way part is the real story — watching them struggle to explain a concept exposed gaps in how *I* was explaining it, and it directly sharpened my own teaching `[VERIFY/REPLACE]`. Helping them made me better at the thing I thought I'd mastered.

That's also why a Developer Advocate role at the Security Lab appeals to me — scaling other people's ability to contribute *is* the job, and it's where I learn the most."

Reflection: "Helping someone teach always teaches me — the questions a learner asks are the best audit of your own understanding."

Theme 7 — Remote & asynchronous collaboration

> Strong, honest grounding: you've built *global, asynchronous* communities and content for decades (myITforum, serialized courses on GitHub/Substack, Microsoft remote). This is a confident draft.

Draft (spoken):

"Async isn't new to me — I was running globally distributed online communities before 'remote work' was a phrase `[VERIFY: myITforum]`, and most of my impact has come from things that work *without* a meeting: serialized courses on GitHub, written series, documentation people consume on their own time and across time zones.

My default is **write it down, in the open.** When I built Must Learn KQL and Must Learn AI Security, the whole point was that someone in another time zone could learn, ask, and contribute without me being online `[VERIFY]`. I treat a good written artifact — a clear issue, a PR description, a versioned doc — as the unit of collaboration.

And I'm deliberate about **when sync is actually worth it.** I reserve real-time for the things writing is bad at: resolving a genuine disagreement, building trust with someone new, or untangling something ambiguous fast. Status, decisions, and teaching should be async and durable — a meeting that could've been a doc just erases the record `[VERIFY: an example where you chose doc-over-meeting or vice-versa]`.

That maps directly to how GitHub and the Security Lab work — distributed, writing-first, research published in the open."

Reflection: "The discipline I've built: default to a durable written artifact, and spend synchronous time only where trust or ambiguity actually needs it."

Theme 10 — A failure / something I'd do differently

> No public failure to anchor to, so this is a **scaffold** — plausible and on-brand, but please `[REPLACE]` with a real one; authenticity matters most here. Lead with the learning, own it, no blame.

Draft (spoken):

"Early on with the Must Learn material `[REPLACE with a real miss if you have one]`, I shipped a section I was proud of that quietly *failed* — engagement dropped and people got stuck `[VERIFY: what actually happened]`. When I dug in, the cause was on me: I'd assumed a level of background the audience didn't have, so I'd skipped the step that actually mattered. I'd optimized for *my* sense of elegance instead of the reader's reality.

What I did about it: I rewrote it from the stuck person's point of view, and — more importantly — I changed my *process*. That's partly why I later built an **assessment** `[VERIFY]` — so I'd get an honest signal about where material was failing instead of trusting my own taste.

What I'd do differently, and now do by default: validate against the real audience *early*, before I'm attached to the polished version. The failure wasn't the bad section; it was waiting too long to find out it was bad."

Reflection: "I stopped trusting 'I think this is good' as evidence — now I build in a way to *measure* whether it worked, and I'd rather find out it's wrong early and cheaply."

Notes

- **Priority to make real:** Themes 3, 5, 10 — a genuine specific memory will beat any scaffold. Even one true detail (a name, a project, a number) transforms these.
- Theme 7 you can largely use as-is once you confirm the `[VERIFY]` anchors.
- With these, all 10 themes now have a draft (1,2,4,6,8,9 well-grounded; 3,5,7,10 scaffolds).
- Say the word and I'll **regenerate the prep packet PDF** so it includes these.

Behavioral & culture workbook

The behavioral interview is **half the loop** and they grade it hard. You can't script it, but you *should* walk in with ~6–8 real stories you can flex across themes. This workbook gives you the structure; **you supply the stories.** Paste rough notes back to me and I'll help shape any of them into tight STAR+R form.

> Format for every answer: **STAR + Reflection**. Spend most of your airtime on **Action → Result → Reflection**, not background. Use **"I"**, not "we" — they want *your* contribution. ~2 minutes per answer.
>
> S — Situation (1–2 sentences of context) T — Task (what *you* were responsible for) A — Action (what *you specifically* did, and *why* — reasoning matters) R — Result (what changed; quantify if you can) **R — Reflection (what you learned and now do differently)**

Build a story bank first (do this before drilling questions)

List 6–8 real experiences. Strong stories are specific, recent-ish, and show *your* decisions. One story can cover several themes — map them so you're never caught without one.

#	Story (1 line)	Themes it can cover	Metric / outcome
1			
2			
3			
4			
5			
6			

Aim to cover: a hard technical decision · a collaboration/disagreement · a time you mentored · a time you owned ambiguity · a failure/learning · an advocacy/teaching win · an OSS or community contribution.

Theme 1 — Motivation: why security, why GitHub, why the Security Lab

They explicitly warn against "GitHub is a great company." Be specific and personal. Prompts: - What's the *origin moment* that pulled you toward security? - Why **open-source** security specifically — what do you believe about it? - Why the **Security Lab** over other security teams? (Use the research brief: the "scale expertise through community" mission, MRVA, query bounties, Taskflow Agent.) - This is a **Developer Advocate** role — connect your love of *teaching/communicating* security to their mission of enabling the community. Your two talks are living proof.

Draft: - Why security: _ - Why open source: _ - Why the Security Lab (name a specific program + why it resonates): _ - Why me / why this role (the DevRel + security combination): _

Theme 2 — Security, open source & technical curiosity

Prompts: a security project you're proud of · an OSS contribution · how you stay current (blogs, communities, conferences) · what you're exploring right now. - Story: _ - What I'm currently exploring & *what I'm doing*

with it: _

Theme 3 — Collaboration & different perspectives

Prompt: a time someone approached a problem differently, you sought the challenge, and your approach *changed*. Emphasize what changed and what you learned — not that you were right. - S/T: _ · A (what *you* did to invite the other view): _ - R + Reflection: _

Theme 4 — Technical problem solving

Use their exact frame: **Problem → Options → Trade-offs → Decision → Outcome → Learning.** They care about *reasoning* as much as the result. Have at least one meaty one. - Problem: _ - Options considered: _ - Trade-offs weighed: _ - Decision + why: _ - Outcome: _ - Learning: _

Theme 5 — Helping & developing others

Prompt: someone you mentored/coached. Strong version = it was **two-way** (you learned too). - Who / situation: _ - What you did: _ - Their outcome + **what you learned from them**: _

Theme 6 — Learning & growth

Prompt: what you're *actively* learning and **what you're doing with it** (concrete > general). - Currently learning: _ - Applied how: _

Theme 7 — Remote & asynchronous collaboration

GitHub is heavily distributed. Prompt: show how you communicate in writing, document, keep people informed, and **decide when sync is actually needed**. - Example of great async work (a doc, an RFC, a well-run issue/PR thread): _ - A time you chose sync *on purpose* and why: _

Theme 8 — Ownership & ambiguity

Their desired arc: **recognized the gap → created clarity → took ownership → brought others along → improved it → applied the learning after.** Avoid blaming others for the ambiguity. - The gap I noticed: _ - How I created clarity: _ - What I owned / how I brought others along: _ - Improvement + what I carried forward: _

Theme 9 — Your definition of excellent work

Have a *philosophy* **and** a real example. Good vs. excellent often = understanding the underlying problem, maintainability, impact, communication, measurable outcomes. - My definition (2–3 sentences): _ - An example where I hit that bar (and how I knew): _

Theme 10 — A failure / something you'd do differently

(Not explicitly listed but almost always asked.) Pick a real one; lead with the learning. - What happened, what I owned, what changed since: _

Dry-run drill

- [ ] Each theme has at least one mapped story.

- [ ] Each story runs ~2 min and is mostly Action/Result/Reflection.
- [ ] Every story has a one-line **Reflection**.
- [ ] You say **"I"** and name a concrete outcome.
- [ ] You've said **"why the Security Lab"** out loud without clichés.
- [ ] Record yourself once; cut the background in half.

Next step: paste me rough notes for Themes 1, 4, and 8 (the highest-signal ones) and I'll turn them into tight STAR+R drafts you can rehearse.

PART C — RESEARCH & QUESTIONS

GitHub Security Lab — research brief

Grounding for "why the Security Lab," your behavioral answers, and your questions. **Re-verify the live numbers on securitylab.github.com the morning of** — stats move. Dates below are as of this prep.

> Reminder: the role is **Developer Advocate** on the Security Lab. The whole loop is effectively a DevRel audition — they're watching *how you teach and tell the story* as much as what you know. Frame your motivation around **enabling the community**, not just doing research yourself.

Mission (say it in your words, don't quote the website)

GitHub Security Lab exists to **secure the world's open-source code by mobilizing the global security community** — not by trying to review everything themselves. The founding framing (2019) named three structural problems, and they're still the best way to explain *why* the Lab exists: - **Scale** — the JavaScript ecosystem alone has 1M+ packages; nobody can hand-audit that. - **Expertise shortage** — developers outnumber security pros by roughly 500:1. - **Coordination** — the world's security experts are scattered across thousands of orgs.

The Lab's answer: **encode security expertise so it scales** (CodeQL queries, models, advisories) and **build a community** that contributes it. That's the thread connecting every program below — and it's exactly the thread in both your talks.

What they actually do (programs to name)

- **CodeQL** — the "code as data" engine. The Lab writes and curates queries, and — key for a DevRel role — helps *the community* write them. (Your Talk 1.)
- **Bug bounty for queries, not just bugs** — a distinctive idea worth calling out: they pay researchers for **CodeQL queries that find a whole *class* of vulnerability**, so one submission protects everyone via code scanning. "Eradicate at scale," not one-off bugs.
- **GitHub Advisory Database / GHSA** — tens of thousands of curated advisories; the backbone of Dependabot alerts. (One of your two original topic options.)
- **Multi-Repository Variant Analysis (MRVA)** — run a CodeQL query across *thousands* of repos at once to find every instance of a pattern ecosystem-wide. The literal embodiment of "find the bug class, not just the bug."
- **CodeQL community packs** & the **CodeQL "Wall of Fame"** — community-contributed queries and recognition for researchers who find CVEs with variant analysis.
- **Security Lab Taskflow Agent** — newer, AI-assisted vulnerability triage (LLM taskflows over CodeQL alerts). Shows where they're heading: AI + static analysis. (A topic option.)
- **The Secure Code Game** — an in-repo, gamified secure-coding learning experience. Pure DevRel/education. (A topic option.)
- **Education & community** — AppSec training, office hours for maintainers, and research published on the blog + Bluesky/Mastodon/LinkedIn.
- **Supply-chain hardening** — the npm + Actions work in your Talk 2 is the Lab/GitHub security org's current frontier.

Headline stats (verify live; use ranges, don't overclaim)

- **~1,200+ vulnerabilities** found by the Lab, with **~900+ CVEs** credited to their work.
- **35,500+** curated security advisories in the GitHub Advisory Database.
- Bug-bounty rewards historically up to the **~$5–6K** range for high-value query work.
- Began at **GitHub Universe 2019**; launched with partners incl. Microsoft, Google, Mozilla, Oracle, Uber, VMware, Trail of Bits, NCC Group, F5, J.P. Morgan. (Sources: securitylab.github.com; github.blog; the 2019 launch post.)

Proof points you can drop into an answer

- CodeQL variant analysis found **RCE in high-profile Java apps (2020)**, which chained into fixing a critical vuln in **Germany's COVID-19 response infrastructure** — a clean "one query → broad real-world impact" story.
- Variant analysis has been pointed at the **Linux kernel and Chromium**.
- The **2026 supply-chain changes** (npm 12 install scripts off, OIDC trusted publishing, safer checkout defaults, Dependabot cooldown) — your Talk 2 — are current and topical.

How "research turns into impact" (their favorite phrase — be ready to riff)

Lab's flywheel: **research a bug → generalize it into a CodeQL query/model → ship it into code scanning + MRVA → the whole ecosystem is now protected → credit/teach the community so they do it too.** Both your talks are instances of this flywheel — say that explicitly.

Why this matters for YOUR two talks (consistency hook they grade)

- Talk 1 (CodeQL) = **detection**, and the "models as data" lever = *encode expertise so it scales* = the Lab's mission in one line.
- Talk 2 (supply chain) = **prevention**, secure-by-default = *protect people who do nothing*.
- Together: **detect + prevent**, and **CodeQL scans both your code and your CI** — one engine, two surfaces. That's your through-line; name it in both talks' openings or closings.

Questions to ask them

Goal: **start a conversation**, not fetch a fact you could Google. Pick 4–6 you genuinely care about; have a couple extra in reserve. Jot a one-line note after each answer — referencing it later ("you mentioned earlier…") is a strong signal. Tailor each to the interviewer's role (researcher vs. manager vs. DevRel).

About the mission & where it's going

1. The Lab's premise is that you can't out-hire the problem — you scale expertise through the community. **Where does that model work best today, and where does it still break down?**
2. With the **Taskflow Agent** and AI-assisted triage, how is the Lab thinking about the line between what the LLM decides and what a human researcher must still own?
3. If you look 12–18 months out, **where does the Lab most want to have more impact** that it can't quite reach yet?

About research → impact (their phrase)

4. When a researcher finds a novel bug class, what does the path from **"interesting finding" to "query shipping in code scanning / MRVA"** actually look like — and where does it get stuck?
5. How do you measure whether a piece of research actually moved the needle for the ecosystem, versus just generating a CVE?

About the Developer Advocate role specifically

6. For this DevRel role, **what does a great first 90 days look like** — more writing/queries, more community/conference presence, or more maintainer enablement?
7. How does the Lab decide **which audiences to invest in** — individual maintainers, security researchers, or enterprise security teams? They need pretty different stories.
8. What's a piece of security tech the team believes is genuinely important but that developers still **don't "get"** — where better advocacy would change adoption?

About the craft & technical decisions

9. CodeQL constantly trades **precision vs. coverage**. How does the team decide when a query is "good enough" to enable by default versus leave opt-in?
10. On the supply-chain work — install scripts off by default, OIDC trusted publishing — **how do you weigh ecosystem friction against security** when changing a default that affects millions of builds?

About the team & how it works

11. GitHub is famously remote/async. **How does the Lab do deep security research asynchronously** — what's written down vs. what still needs a synchronous room?
12. When the team disagrees on a technical call, how does that usually get resolved?

Closing-style question (good last one)

13. **What separates someone who's good in this role from someone who's exceptional?**

Avoid: "What does the Security Lab do?", "Is CodeQL free?", benefits/logistics (save for recruiter), anything answered on the first page of their site.